Lesson 2.2: Task decomposition

Claude Certified Associate – Foundations (CCAO-F) · Module 2: Prompting & Task Execution · Lesson 2 · 19:44

Description

Real requests are rarely one job. "Build a glossary from these notes" sounds like one thing, but it hides five: find every term, merge the duplicates, define each one, check them, and lay it all out. This lesson teaches task decomposition (breaking a request into smaller steps, each with one clear goal) and prompt chaining (separate prompts in order, where each answer feeds the next). It shows how a chain differs from numbered steps inside one prompt and from Claude's own thinking, and when one prompt is the better choice. You see where the guidance comes from: the official exam guide, Anthropic's official prep course, the Claude blog, the Claude Platform Docs and the Help Center. Then one real job runs in Claude Desktop twice: a glossary of every key term from Lessons 1.1 to 2.1, first as one good prompt, then as a chain of three steps with a checkpoint where you make the decisions. The lesson ends with one exam-style scenario, reasoned out loud, and the cue to this lesson's quick check.

Learning outcomes

By the end of this lesson you can:

1. Explain task decomposition and prompt chaining, and how each step's output feeds the next.
2. Split a real request into a short chain, with a checkpoint between steps.
3. Decide when to split, and when one prompt (and Claude's own thinking) is enough.

Chapters

Time	Chapter
00:00	Preview and outcomes
00:35	Why it matters
01:27	The idea: one prompt to a chain
03:29	Straight from the source
05:43	Hands-on: one prompt vs a chain
17:46	Exam angle
19:01	Recap and quick check

The idea

One prompt, five jobs. Ask for everything at once and Claude does it in one pass. Often that's fine. But every decision in the middle, like which terms to merge, happens out of sight. If one goes wrong, it flows into everything after it, and all you see is the end.

From one prompt to a chain. Task decomposition breaks the one big block into subtasks, each with one clear goal. In this lesson there are three: find the terms; decide what to merge, and define; check. Prompt chaining connects them: you send step 1 and read the answer; that answer becomes the input for step 2, and step 2's answer feeds step 3. The answers in the middle are intermediate outputs. Between two steps you can read one, fix it, and only then continue. This course calls that moment a checkpoint. A mistake caught there never reaches the end.

Three ways to break a task down (this course's grouping, built from Anthropic's guidance):

Way	Where it happens	Can you steer it midway?
Numbered steps inside one prompt	One message. The Docs suggest it when the order or completeness of steps matters.	No
Thinking	Inside one reply: Claude breaks the problem down for itself before it answers. With Sonnet 5.5 in the app, thinking is always on.	No
A chain across messages	Several messages: you see each step's output.	Yes

When not to split (synthesized from the blog and the Docs):

- One simple job: just ask.
- You won't actually read the in-between answers: a chain only adds waiting.
- Every part needs the whole picture (like finding themes across a stack of survey comments): cutting it up can make things worse.
- The Docs' anchor: Claude handles most multistep reasoning internally. Split when you need to see and steer in between.

When you do split, four moves (this course's recipe):

1. List the jobs as verbs.
2. Order them, so each output feeds the next.

3. Give each step one clear goal.
4. Put your checkpoint where a mistake would spread furthest.

A common last step is a check: Claude reviews the draft against your criteria, then fixes it (the Docs' self-correction pattern: draft, review against criteria, refine).

Straight from the source (what the lesson shows)

- **Exam guide, page 4:** Domain 1, Prompting and Task Execution (14% of scored items). Its second bullet: "Apply task decomposition techniques to structure complex requests". The guide says exam items are written against these objectives. This course labels the bullet task 1.2; the guide lists its task statements as unnumbered bullets, so 1.2 is our label, not the guide's.
- **Exam guide, page 6, How to Prepare:** "Practice structuring prompts, decomposing tasks, and iterating to improve outputs."
- **Anthropic's official prep course, Prompting & Task Execution:** learning objective 2, "Apply task decomposition techniques to structure complex, multi-part requests".
- **Claude blog, Best practices for prompt engineering:** prompt chaining "breaks down complex tasks into smaller sequential steps with separate prompts. Each prompt handles one stage, and the output feeds into the next instruction." You can run it by hand, sending each prompt after reading the response, which is what the hands-on does. When to chain: a complex request, work in stages, inconsistent results from one prompt, and when checking the in-between result adds value. The trade-off: more waiting, often much better accuracy. Its decision list: a simple task gets the core techniques only; a complex one, consider breaking it down.
- **Claude Platform Docs, Prompting best practices:** Claude handles most multistep reasoning internally; explicit chaining is still useful when you need to inspect intermediate outputs. The most common chain is self-correction: draft, review against criteria, refine.
- **Claude Help Center, model, effort and thinking settings:** thinking gives Claude more time to break down problems and plan before it responds. With Sonnet 5.5 it can't be turned off. A chain adds what thinking can't: your eyes between the steps.

Key terms

- **Task decomposition:** breaking a complex request into smaller steps, each with one clear goal.
- **Subtask:** one of those smaller steps.
- **Prompt chaining:** separate prompts in order, where each answer feeds the next prompt.
- **Intermediate output:** the answer from a middle step: input for the next step, and something you can check.
- **Checkpoint:** the moment between steps where you read, correct, then continue. This is the course's term; Anthropic says "inspect intermediate outputs" and "intermediate validation".
- **Thinking:** Claude breaking a problem down for itself before it replies. It's always on with Sonnet 5.5 in the app.

What happened in the hands-on

Claude Desktop on Linux, Chat view, Sonnet 5.5 at Medium, recorded on 2026-10-05 on a real account. Every prompt went into a new **incognito chat** (Lesson 1.4), which isn't saved and uses no memory. The material is real: the published lesson notes of Lessons 1.1, 1.2, 1.3, 1.4 and 2.1 in one file, almost 12,000 words, with 29 key-term entries. The job: a glossary of every key term for this course's student kit. The exact prompts, the attached notes and Claude's replies are in the student kit. Four long waits while Claude worked are shortened with labelled time cuts (+43 s, +28 s, +41 s, +60 s), said aloud; nothing you type or read is cut.

Version 1: the whole job in one prompt (one generation). A good prompt, built the way Lesson 2.1 taught, with a label on each line: task, context, audience, constraints (merge duplicates, handle words with two meanings, under 20 words, only the notes, check every definition), format (one alphabetical table: Term, Definition, Lesson).

1. While it worked, the gray status line showed the decisions happening in one pass: "Merging overlapping terms like model and context definitions", "Deciding which near-duplicate terms to keep separate". You can glimpse them, but you can't step in.
2. **The result was good.** 27 terms in one alphabetical table. It merged the Lesson 1.1 meaning of context with context window (calling it a judgment call), gave the Lesson 2.1 meaning (the background you put in a prompt) its own row, kept hand-off and hand-off summary apart, merged model and model tier, and kept the small condition that automatic context management happens only on paid plans with code execution on.
3. It ended by saying it checked every definition against the notes. Every definition really is under 20 words (counted), but that check happened out of sight, so you take it on trust.

Version 2: the same job as a chain (three generations), in a fresh incognito chat with the same file attached the same way. Only the way of asking changed.

4. **Step 1, find.** List every key term with its lesson and its definition quoted exactly from the notes; flag terms that might mean the same thing and words used with two meanings. One sentence matters: "Don't write the glossary yet." (The Docs' notes for Sonnet 5.5: if you want a list first, say so.) The status line now read "noting" and "mapping", not merging or deciding.
5. **The intermediate output:** 29 entries, each quote matching the notes (all 29 checked). Then the flags: six pairs that might mean the same thing (such as context and context window, model and model tier) and five words with two meanings (context, hand-off, memory, limit, model). It ended: "Say when you're ready for the glossary." It waited, exactly as asked.
6. **The checkpoint.** Three rules, applied by the learner, not by Claude: same meaning, merge under the more precise name; same word with two meanings, two labelled entries; related but different, keep both and make the difference clear.
7. **Step 2, decide and define.** The decisions went in as the first line (merge Model into Model tier; merge the Lesson 1.1 Context into Context window; keep the Lesson 2.1 Context as "Context (in a prompt)"; keep Hand-off and Hand-off summary apart; "The other flags

need no change", added because Claude raised more flags than planned). Then the task, same audience, same limit, same table format. The list from step 1 was the input, still in the chat, so there was nothing to paste.
8. **The draft:** 27 entries, following every decision. It noted three details it dropped to stay under 20 words. But it silently dropped one more: the paid-plans, code-execution condition on automatic context management.
9. **Step 3, check.** Check the glossary against the quotes from step 1: does each entry keep the meaning, and is it under 20 words? List only the failures and why, then the corrected table.
10. **The check, out in the open:** "Only one entry fails: Automatic context management." The length was fine; the meaning had lost the condition. The other 26 passed. The corrected entry puts the condition back, in 19 words. At the very end it owned up to two more small omissions it hadn't mentioned (the origin labels on two entries, and Effort's "from Low to Max" range) and offered to put them back.
11. **Your own check (Discernment, Lesson 0.2).** A self-check is still Claude grading its own work, so the lesson scrolls back to the step-1 quote for automatic context management and compares: same meaning, 19 words. Context window checks out too, minus the "working memory" phrase it said it dropped. The chain made this easy: the quotes were already in step 1.
12. **Side by side.** On quality it's close to a tie, and the one prompt even kept the condition the draft dropped. What the chain gave was control (the learner made the merge calls), visibility (the list and the flags), and a check you could see, which caught a real slip. The price: three waits instead of one, and more typing. Worth it here; for a quick, simple job, it isn't.

The exam link

Domain 1, Prompting and Task Execution (14% of scored items). This lesson serves the bullet this course labels **1.2**: "Apply task decomposition techniques to structure complex requests". The guide lists its task statements as unnumbered bullets; the number 1.2 is this course's label, not the guide's.

The lesson's practice scenario (original, written for this course, not from the exam). A people-operations lead pastes sixty anonymous survey comments and asks Claude, in one long prompt, to sort them into themes, count each theme, pick the top three and write a memo to leadership. The memo leads with workload, but skimming the comments, scheduling looks far more common. She can't tell where it went wrong. What's the most appropriate next step? A: run the same prompt on the most capable model. B: add "think step by step and be accurate". C: split it: sort and count the themes first, check the counts against the comments, then pick the top three and draft from the checked list. D: send each comment to its own chat for a label, and combine them herself. The answer is **C**. Ask where the mistake could have entered: the counting, and it was invisible. A spends a bigger model on the same blind spot. B asks for more thinking, but thinking happens inside one reply, and with Sonnet 5.5 it's already on. D splits too far: themes only make sense when all sixty comments are compared together. C puts a checkpoint right where the error entered.

Quick check

Before you move on, take this lesson's quick check: a few short questions, a couple of minutes. The exam-style question in this lesson, the quick checks, the module quizzes and the practice exams are all original, written for this course against the official blueprint. They are never copies of the real exam.

Sources (all checked 2026-10-05)

- **Anthropic, *Claude Certified Associate – Foundations Exam Guide*.** Version 1.0, effective July 2026. §6 Domain 1 and detailed objectives (p. 4), §7 How to Prepare (p. 6). Official download: https://everpath-course-content.s3-accelerate.amazonaws.com/instructor/6nizmqk8tpzpfjvt6qmmav7rh/public/1783542847/Claude+Certified+Associate+%E2%80%93+Foun
- **Anthropic's official CCAO-F prep course,** Course 2, *Prompting & Task Execution* (public course page: learning objective 2; path page summary).
- **Claude blog,** *Best practices for prompt engineering* (November 10, 2025: prompt chaining, when to use it and its trade-off, choosing the right techniques): https://claude.com/blog/best-practices-for-prompt-engineering
- **Claude Platform Docs,** *Prompting best practices* (be clear and direct; leverage thinking; chain complex prompts): https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices
- **Claude Platform Docs,** *Prompting Claude Sonnet 5.5* (if you want a list or a plan first, say so): https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/prompting-claude-sonnet-5-5
- **Claude Help Center,** *Change the model, effort, and thinking settings* (use thinking; it can't be turned off with Sonnet 5.5): https://support.claude.com/en/articles/8664678-change-the-model-effort-and-thinking-settings
- **Claude Academy, AI Fluency: Framework and foundations,** *Effective prompting techniques* (break complex tasks into steps; ask the AI to think first): https://academy.claude.com/courses/ai-fluency-framework-foundations/effective-prompting-techniques
- **The app itself:** Claude Desktop on Linux, Chat view, Sonnet 5.5 Medium, incognito chats, observed 2026-10-05. Menus, labels and status lines can change between versions.

The checkpoint, the three ways to break a task down, the "when not to split" questions and the four-move recipe are this course's own teaching tools, built from the sources above. The hands-on material is real: this course's own published lesson notes. The glossary and every decision at the checkpoint are the course author's.

What you see in the lesson is a real account. Settings, Account and Memory were never opened, and Claude's Thinking section was never expanded. A usage notice that Claude Desktop showed during recording is blurred.